

Spring IoC and DI

In **Spring Framework**, **IoC (Inversion of Control)** and **DI (Dependency Injection)** are core concepts used to build loosely coupled and maintainable applications.

1 Inversion of Control (IoC)

✓ Definition:

IoC means that the control of creating objects and managing their lifecycle is transferred from the application code to the Spring container.

◆ Without IoC (Traditional way)

Java

```
class Car {  
    Engine engine = new Engine(); // tightly coupled  
}
```

Here:

- **Car** creates its own **Engine**
- Car is tightly coupled with Engine
- Hard to test and maintain

◆ With IoC (Using Spring)

Java

```
class Car {  
    private Engine engine;    public Car(Engine engine) {  
        this.engine = engine;  
    }  
}
```

Now:

- Car does NOT create Engine
- Spring container creates Engine and injects it
- Car is loosely coupled

👉 **Spring IoC container** manages:

- Object creation
- Dependency management
- Lifecycle of beans

2 Dependency Injection (DI)

✓ **Definition:**

DI is a design pattern used to implement IoC.

It means:

The dependencies of a class are provided (injected) from outside instead of the class creating them itself.

Types of Dependency Injection in Spring

1 Constructor Injection (Recommended)

Java

```
@Component
class Car {    private Engine engine;    @Autowired
    public Car(Engine engine) {
        this.engine = engine;
    }
}
```

- ✓ Best practice
- ✓ Makes object immutable
- ✓ Easier for unit testing

2 Setter Injection

Java

```
@Component
class Car {    private Engine engine;    @Autowired
    public void setEngine(Engine engine) {
        this.engine = engine;
    }
}
```

- ✓ Used for optional dependencies

3 Field Injection (Not Recommended)

Java

```
@Component
class Car {    @Autowired
    private Engine engine;
}
```

- ✗ Hard to test
- ✗ Breaks encapsulation

How Spring IoC Container Works

1. Reads configuration (XML / Java Config / Annotations)
2. Creates beans
3. Resolves dependencies
4. Injects dependencies
5. Manages bean lifecycle

IoC vs DI (Difference)

IoC	DI
Principle	Design pattern
Inverts object creation control	Injects required dependencies
Implemented by Spring container	One way to achieve IoC

👉 Simple way to remember:

- **IoC = "Who controls object creation?" → Spring**
- **DI = "How are dependencies provided?" → Injection**

Why IoC & DI Are Important?

- ✓ Loose coupling
- ✓ Easier testing (mocking)
- ✓ Better maintainability
- ✓ Better modular design

1 Why is Constructor Injection a Best Practice?

◆ Reason:

It forces you to provide all required dependencies at object creation time.

```
Java

@Component
class Car {
    private final Engine engine;    public Car(Engine engine) {
        this.engine = engine;
    }
}
```

👉 What this ensures:

- Object is **always in a valid state**
- No chance of **null** dependency
- No partially initialized objects

✗ Compare with Setter Injection

Java

```
Car car = new Car();  
car.setEngine(new Engine()); // may forget this!
```

⚠ Risk:

Someone might forget to set dependency → runtime error

👉 That's why constructor injection is considered **safe + clean + predictable**

2 How it makes object Immutable

◆ Key idea:

If you use:

- **final** fields
- Constructor initialization only

Then the object **cannot change after creation**

Java

```
class Car {  
    private final Engine engine;    public Car(Engine engine) {  
        this.engine = engine;  
    }  
}
```

👉 Now:

- `engine` cannot be reassigned
- No setter → no modification later

✅ Benefits:

- Thread-safe (important in Spring singleton beans)
- No accidental changes
- Easier debugging

❌ Setter Injection Problem:

Java

```
car.setEngine(new DieselEngine());  
car.setEngine(new PetrolEngine()); // can change anytime!
```

👉 Object behavior becomes unpredictable

3 Why it is Easier for Unit Testing

This is the most practical advantage.

◆ With Constructor Injection:

You can directly pass mock objects:

Java

```
Engine mockEngine = new FakeEngine(); Car car = new Car(mockEngine);
```

- 👉 No Spring needed
- 👉 No container needed
- 👉 Pure Java testing

◆ With Field Injection:

Java

```
@Component
class Car {
    @Autowired
    private Engine engine;
}
```

👉 Problem:

- You cannot inject dependency manually

- You must use Spring context or reflection

♦ With Setter Injection:

Java

```
Car car = new Car();  
car.setEngine(mockEngine);
```

👉 Works, but:

- Dependency is optional (not enforced)
- Test may not reflect real behavior

🔥 Interview Summary (Use This)

👉 If interviewer asks:

Why constructor injection is preferred?

You say:

Constructor injection is preferred because it ensures all required dependencies are provided at object creation time, making the object immutable and preventing null states. It also makes unit testing easier since dependencies can be passed manually without requiring the Spring container.

💡 One-Line Memory Trick

- **Constructor** → **Mandatory + Safe + Testable**
- **Setter** → **Optional + Flexible**
- **Field** → **Avoid**

If you want next level clarity, I can:

- Show real Spring Boot example

- Give tricky interview questions on DI (very common)
- Show when setter injection is actually better (edge cases)

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#).